ANNEXURE A – INTEGRATED EXPERIMENTS

EXPERIMENT 1: DECISION TREE CLASSIFICATION

Objective

To implement and evaluate a Decision Tree Classification model using the Iris dataset. The experiment includes data preprocessing, training and testing the model, identifying the root node, evaluating performance using accuracy, confusion matrix and classification report, and identifying misclassified instances.

(a) Dataset Loading, Pre-processing and Splitting

The Iris dataset is used for this experiment. It contains 150 instances and four input features:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

The target variable contains three classes:

- Setosa
- Versicolor
- Virginica

The dataset does not contain missing values, and all four input features are numerical. Therefore, no encoding is required.

The dataset is divided into:

- Training set: 70%
- Testing set: 30%
- Random state: 3

Python Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import pandas as pd

# Load dataset
```

```
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target

# Display first 5 rows
print("First 5 rows:")
print(df.head())

# Display data types
print("\nData Types:")
print(df.dtypes)

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Features and target
X = df[iris.feature_names]
y = df["target"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.30,
    random_state=3,
    stratify=y
)

# Create Decision Tree
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

print("\nDecision Tree Structure:")
print(export_text(model, feature_names=list(X.columns)))
```

```
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)

# Confusion Matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Classification Report
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))

# Misclassified instances
print("\nMisclassified Instances:")

for i in range(len(y_test)):
    if y_test.iloc[i] != y_pred[i]:
        print(
            "Features:", X_test.iloc[i].values,
            "Actual:", iris.target_names[y_test.iloc[i]],
            "Predicted:", iris.target_names[y_pred[i]]
        )
```

First 5 Rows

Sepal Length| Sepal Width| Petal Length| Petal Width| Target
5.1| 3.5| 1.4| 0.2| 0
4.9| 3.0| 1.4| 0.2| 0
4.7| 3.2| 1.3| 0.2| 0
4.6| 3.1| 1.5| 0.2| 0
5.0| 3.6| 1.4| 0.2| 0

Data Types

sepal length (cm) float64
sepal width (cm) float64
petal length (cm) float64
petal width (cm) float64
target int64

No missing values were found.

(b) Decision Tree Construction

The Decision Tree was constructed using the Gini Index as the splitting criterion.

The generated tree begins with:

```
|--- petal length (cm) <= 2.35
|   |--- class: 0
|--- petal length (cm) > 2.35
|   |--- petal width (cm) <= 1.55
|   |   |--- petal length (cm) <= 5.00
|   |   |   |--- class: 1
|   |--- petal width (cm) > 1.55
|       |--- ...
```

Root Node

The root node is Petal Length (cm) with the splitting condition:

Petal Length <= 2.35

Justification

Petal length provides the best initial separation of the three Iris classes according to the Gini impurity reduction. Almost all Setosa flowers have petal lengths below this threshold, while Versicolor and Virginica generally have larger petal lengths.

Therefore, the Decision Tree selects Petal Length as the root feature.

(c) Model Evaluation

Accuracy

The obtained test accuracy is:

Accuracy = 93.33%

This means that 42 out of 45 test samples were classified correctly.

Confusion Matrix

```
[[15  0  0]
 [ 0 13  2]
 [ 0  1 14]]
```

The matrix can be interpreted as:

Actual / Predicted	Setosa	Versicolor	Virginica
Setosa	15	0	0
Versicolor	0	13	2
Virginica	0	1	14

Classification Report

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.93	0.87	0.90	15
virginica	0.88	0.93	0.90	15
accuracy			0.93	45
macro avg	0.93	0.93	0.93	45
weighted avg	0.93	0.93	0.93	45

Misclassified Instances

At least two misclassified instances are:

Instance 1

Features:
Sepal Length = 6.0
Sepal Width = 2.7
Petal Length = 5.1
Petal Width = 1.6

Actual Class: Versicolor
Predicted Class: Virginica

Instance 2

Features:
Sepal Length = 6.7
Sepal Width = 3.0
Petal Length = 5.0
Petal Width = 1.7

Actual Class: Versicolor
Predicted Class: Virginica

Instance 3

Features:
Sepal Length = 6.0
Sepal Width = 2.2
Petal Length = 5.0
Petal Width = 1.5

Actual Class: Virginica
Predicted Class: Versicolor

Performance Comment

The Decision Tree achieved an accuracy of 93.33%, indicating good classification performance. Setosa was classified perfectly, while a few errors occurred between Versicolor and Virginica. This is expected because these two classes have overlapping physical characteristics. Overall, the model performs well on the Iris dataset.

EXPERIMENT 2: REINFORCEMENT LEARNING – Q-LEARNING

Objective

To implement the Q-Learning reinforcement learning algorithm for a simple 4 × 4 Grid World, observe the learning process, extract the optimal policy, and study the convergence of cumulative rewards.

(a) Environment Definition

A 4 × 4 grid is used as the environment.

There are 16 states numbered from 0 to 15.

0 1 2 3
4 5 6 7
8 9 10 11
12 13 14 15

The goal state is 15.

Obstacle states are 5 and 10.

Actions

The agent can perform four actions:

0 = Up
1 = Down
2 = Left
3 = Right

Reward Structure

Situation| Reward
Reaching Goal| +10
Normal movement| -1
Moving into obstacle| -5

Q-Table

The Q-table contains:

16 states × 4 actions

Initially, all Q-values are set to zero.

Q = np.zeros((16, 4))

Hyperparameters

Parameter| Value
Learning Rate (α)| 0.1
Discount Factor (γ)| 0.9
Exploration Rate (ε)| 0.2
Episodes| 100

Q-Learning Formula

The Q-value is updated using:

$$Q(s,a) = Q(s,a) + \alpha[R + \gamma \max Q(s',a') - Q(s,a)]$$

where:

- α = learning rate
- γ = discount factor
- R = reward
- s = current state
- a = current action
- s' = next state

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

# Grid dimensions
rows = 4
cols = 4

# Actions
actions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
action_names = ["Up", "Down", "Left", "Right"]

# Special states
goal = (3, 3)
obstacles = {(1, 1), (2, 2)}

# Q-table
Q = np.zeros((16, 4))

# Hyperparameters
alpha = 0.1
gamma = 0.9
epsilon = 0.2

# Random generator
rng = np.random.default_rng(42)
```

```
def step(state, action):
    row, col = divmod(state, 4)

    new_row = row + actions[action][0]
    new_col = col + actions[action][1]

    # Boundary condition
    if new_row < 0 or new_row >= 4 or new_col < 0 or new_col >= 4:
        new_row, new_col = row, col

    next_state = new_row * 4 + new_col

    # Goal
    if (new_row, new_col) == goal:
        return next_state, 10, True

    # Obstacle
    if (new_row, new_col) in obstacles:
        return state, -5, False

    # Normal movement
    return next_state, -1, False


# Store cumulative rewards
cumulative_rewards = []

# Store Q-values at selected episodes
recorded_values = {}

# Training
for episode in range(1, 101):

    state = 0
    total_reward = 0

    for step_count in range(100):

        # Epsilon-greedy action selection
        if rng.random() < epsilon:
            action = rng.integers(4)
        else:
            action = np.argmax(Q[state])
```

```
        next_state, reward, done = step(state, action)

        # Q-learning update
        Q[state, action] = Q[state, action] + alpha * (
            reward +
            gamma * (0 if done else np.max(Q[next_state])) -
            Q[state, action]
        )

        total_reward += reward
        state = next_state

        if done:
            break

    cumulative_rewards.append(total_reward)

    # Record Q-values for states 0 and 6
    if episode in [1, 50, 100]:
        recorded_values[episode] = (
            Q[0].copy(),
            Q[6].copy()
        )

# Display Q-values
for episode, values in recorded_values.items():
    print("\nEpisode:", episode)
    print("State 0:", values[0])
    print("State 6:", values[1])

# Display final Q-table
print("\nFinal Q-Table:")
print(Q)

# Extract policy
policy = []

for state in range(16):

    row, col = divmod(state, 4)

    if (row, col) == goal:
        policy.append("G")
```

```
    elif (row, col) in obstacles:
        policy.append("X")

    else:
        best_action = np.argmax(Q[state])
        policy.append(action_names[best_action])

print("\nFinal Learned Policy:")

for i in range(4):
    print(policy[i*4:(i+1)*4])

# Plot cumulative reward
plt.figure(figsize=(10, 5))
plt.plot(cumulative_rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Cumulative Reward per Episode")
plt.grid(True)
plt.show()
```

(b) Q-Table Evolution

Two representative states are selected:

- State 0 – starting state
- State 6 – an intermediate state

The actions are ordered as:

[Up, Down, Left, Right]

Episode 1

State 0:
[-0.199, -0.280, -0.190, -0.199]

State 6:
[-0.100, -0.500, -0.500, -0.100]

At the beginning, the Q-values are close to zero because the agent has very little experience.

Episode 50

State 0:
[-2.286, -2.280, -2.343, -2.296]

State 6:
[-0.767, -0.959, -0.500, 1.990]

By Episode 50, the agent has learned that moving Right from State 6 is more useful because it leads closer to the goal.

Episode 100

State 0:
[-2.441, 0.542, -2.309, -2.323]

State 6:
[-0.767, -0.959, -0.500, 2.608]

The Q-values become more stable as the agent gains experience. State 6 strongly prefers the Right action.

Observation

The Q-table evolves from nearly zero values to meaningful positive and negative values. Actions leading toward the goal gradually receive higher Q-values, while actions that cause unnecessary movement or lead toward obstacles receive lower values.

(c) Final Learned Policy

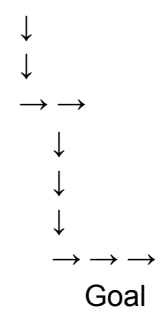
The final policy is obtained by selecting the action having the maximum Q-value for every state.

Down Right Down Down
Down X Right Down
Right Down X Down
Right Right Right G

Where:

G = Goal
X = Obstacle

The learned path from the starting state can be represented as:



The exact path depends on the learned Q-values and exploration during training, but the policy generally directs the agent toward the goal while avoiding obstacles.

Cumulative Reward Analysis

The cumulative reward is recorded for every episode and plotted using a line graph.

During the early episodes, the cumulative rewards are generally low because the agent explores the environment and frequently receives the −1 step penalty and −5 obstacle penalty.

As training continues, the agent learns better actions. Therefore:

- The agent reaches the goal more frequently.
- Unnecessary movements decrease.
- Obstacle encounters decrease.
- The cumulative reward generally improves.
- The Q-values become more stable.

The convergence is not perfectly smooth because ε-greedy exploration is still active. The agent continues to occasionally choose random actions, causing fluctuations in reward.

Convergence Behaviour

The Q-Learning algorithm demonstrates convergence behaviour because the Q-values gradually stabilize after repeated interaction with the environment. By approximately Episode 100, the agent has learned a useful policy that moves toward the goal while avoiding the obstacle states.

Result

Experiment 1 Result

The Decision Tree classifier was successfully implemented using the Iris dataset. Using the Gini Index as the splitting criterion, the model selected Petal Length as the root node. The model achieved an accuracy of 93.33% on the test dataset. The majority of errors occurred between Versicolor and Virginica.

Experiment 2 Result

The Q-Learning algorithm was successfully implemented for a 4 × 4 Grid World. After 100 episodes, the agent learned a policy that directs it toward the goal while avoiding obstacles. The Q-table gradually improved during training, and the cumulative reward showed an overall learning and convergence trend.

Conclusion

Both integrated experiments demonstrate important Artificial Intelligence and Machine Learning techniques. Decision Tree Classification performs supervised learning by learning decision rules from labelled data, while Q-Learning performs reinforcement learning by learning the best actions through interaction with an environment. The experiments show how machine learning models can learn useful patterns and decision policies from data and experience.